

**Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Departamento de Ciências de Computação
Disciplina de Algoritmos e Estruturas de Dados II (SCC0503)**

Docente

Profa. Dra. Cristina Dutra de Aguiar
cdac@icmc.usp.br

Monitores

Gustavo Ramos Santos Pires

gustavo.rspires@usp.br ou telegram: @darrocaxd

João Gabriel Pieroli da Silva

joaogabrielpieroli@usp.br ou telegram: @bielcomp24

Pedro Lunkes Villela

pedrolunkesvillela@usp.br ou telegram: @pedrolunkes

Renan Banci Catarin

renanbcatarin@usp.br ou telegram: @Reckat

Renan Trofino Silva

renan.trofino@usp.br ou telegram: @renan823

Vinícius Souza Freitas

vininicius@usp.br ou telegram: @Vininicius

Trabalho Prático 2

Este trabalho tem como objetivo aprofundar conceitos relacionados a grafos.

*O trabalho deve ser feito por **2 alunos da mesma turma**. Os alunos devem ser os mesmos do trabalho introdutório e do trabalho prático 1. Quaisquer mudanças devem ser devidamente informadas. A solução deve ser proposta exclusivamente pelo(s) aluno(s) com base nos conhecimentos adquiridos nas aulas. Consulte as notas de aula e o livro texto quando necessário.*

Programa

Descrição Geral. Implemente um programa em C/ C++ por meio do qual o usuário possa obter dados de um arquivo binário de entrada, gerar um grafo direcionado a partir deste e realizar investigações interessantes dentro do contexto de estações e linhas do metrô e da CPTM (Companhia Paulista de Trens Metropolitanos) da região metropolitana da cidade de São Paulo (SP).

Importante. A definição da sintaxe de cada comando bem como sua saída devem seguir estritamente as especificações definidas em cada funcionalidade. Para especificar a sintaxe de execução, considere que o programa seja chamado de “programaTrab”. Essas orientações devem ser seguidas uma vez que a correção do funcionamento do programa se dará de forma automática. De forma geral, a primeira entrada da entrada padrão é sempre o identificador de suas funcionalidades, conforme especificado a seguir.

Modularização. É importante modularizar o código. Trechos de programa que aparecerem várias vezes devem ser modularizados em funções e procedimentos.

Descrição Específica. O programa deve oferecer as seguintes funcionalidades:

[10] Permita a recuperação dos dados, de todos os registros, armazenados em arquivos de dados no formato binário e a geração de um grafo contendo esses dados na forma de um conjunto de vértices V e um conjunto de arestas A . Os arquivos de dados no formato binário devem seguir os mesmos formatos dos arquivos de dados gerados no trabalho prático introdutório e no primeiro trabalho prático. Registros marcados como logicamente removidos não devem ser inseridos no grafo. O grafo deve ser um grafo direcionado ponderado e deve representar as estações e as linhas que ligam essas estações com as suas respectivas distâncias.

A representação do grafo deve ser, obrigatoriamente, na forma de listas de adjacências. As listas de adjacências consistem tradicionalmente em um vetor de $|V|$ elementos que são capazes de apontar, cada um, para uma lista linear, de forma que o i -ésimo elemento do vetor aponta para a lista linear de arestas que são adjacentes ao vértice i . Cada elemento do vetor deve representar o **nome de uma estação**. Um exemplo de nome de estação é “Luz”. Os vértices do vetor devem ser ordenados de forma crescente de acordo com o nome da estação. Note que, segundo a especificação do trabalho prático, se duas ou mais estações têm o mesmo nome, elas são consideradas a mesma estação.

Cada elemento da lista linear representa uma aresta entre duas estações e deve armazenar: (i) o nome da próxima estação; (ii) a distância para a próxima estação; e (iii) os nomes de linha associados. Considerando o elemento do vetor com nome de

estação igual a “Luz”, um exemplo de elemento da lista linear referente a essa estação é: (i) nome da próxima estação = “Sao Bento”; (ii) distância para a próxima estação = 762; e (iii) nome da linha = “Azul”. Neste primeiro exemplo, existe apenas um único registro no arquivo de dados de forma que $aresta(u, v) = (Luz, Sao\ Bento)$. Ainda considerando o elemento do vetor com nome de estação igual a “Luz”, outro exemplo de elemento da lista linear é: (i) nome da próxima estação = “Bras”; (ii) distância para a próxima estação = 2310; e (iii) nomes das linhas = “Coral”, “Jade”, “Rubi”. Neste segundo exemplo, existem três registros no arquivo de dados de forma que $aresta(u, v) = (Luz, Bras)$.

Os elementos que devem ser inseridos nas listas lineares devem ser aqueles que atendem aos seguintes requisitos:

- O elemento representa uma aresta entre um par ($codEstacao$, $codProxEstacao$), sendo que o nome da estação referente a $codProxEstacao$ deve ser obtido fazendo-se uma pesquisa no arquivo de dados. Neste caso, a distância entre as estações e o nome da linha encontram-se no registro do arquivo de dados referente a $codEstacao$.
- O elemento representa uma aresta entre um par ($codEstacao$, $codEstacaoIntegra$), sendo que o nome da estação referente a $codEstacaoIntegra$ deve ser obtido fazendo-se uma pesquisa no arquivo de dados. A aresta somente deve ser inserida se o nome da estação referente a $codEstacao$ for diferente do nome da estação referente a $codEstacaoIntegra$. Neste caso, a distância entre as estações deve armazenar o valor zero e o nome da linha deve armazenar o valor “Integração”.

Os elementos de cada lista linear devem ser ordenados de forma crescente de acordo com o nome da estação (quer o nome represente uma próxima estação ou uma estação de integração). Em situações nas quais existem vários nomes de linhas armazenados em um mesmo elemento (ou seja, se houver empate), a ordenação deve ser feita de forma crescente de acordo com os nomes das linhas.

Entrada do programa para a funcionalidade [10]:

10 arquivoEntrada.bin

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório.

Saída caso o programa seja executado com sucesso:

A saída deve ser exibida na saída padrão da seguinte forma. Em cada linha, deve ser mostrado primeiro o elemento do vetor na posição i e depois a lista linear correspondente. Os elementos da lista linear devem ser exibidos de forma crescente de acordo com o nome da estação. Quando necessário, os nomes das linhas devem ser exibidos de forma crescente. Deve haver uma vírgula e um espaço em branco entre cada saída mostrada na saída padrão.

Exemplo com metadados:

```
nomeEstacao1, nomeProxEstacao11, distProxEstacao11, nome(s)Linha(s)11,  
..., nomeProxEstacao1n, distProxEstacao1n  
...  
nomeEstacaom, nomeProxEstacaom1, distProxEstacaom1, nome(s)Linha(s)m1,  
..., nomeProxEstacaomp, distProxEstacaomp
```

Mensagem de saída caso algum erro seja encontrado:

Falha na execução da funcionalidade.

Exemplo de execução (são mostrados apenas alguns elementos):

```
./programaTrab  
10 estacao.bin  
Clinicas Consolacao 1063 Verde  
...  
Luz Bras 2310 Coral Jade Rubi Republica 1257 Amarela Sao Bento 762  
Azul  
...
```

[11] Determine o caminho mais curto entre uma estação origem e uma estação destino, ambas definidas como parâmetro de entrada. Deve ser listado na saída padrão o caminho mais curto. Considere que, durante a execução do algoritmo, qualquer decisão a respeito da ordem de inserção ou análise dos vértices/arestas deve ser feita da seguinte forma: (i) em situação de empate na escolha de vértices, deve ser escolhido o vértice cujo valor do nome da estação seja o menor; e (ii) em situação de empate entre o peso de duas arestas (u_1, v_1) e (u_2, v_2) , deve ser escolhida a aresta cujo valor de u seja o menor. Caso haja situação de empate entre u_1 e u_2 , deve ser escolhida a aresta cujo valor de v seja o menor. Todos os índices dos vetores e listas devem ser ordenados de forma crescente de acordo com o nome da estação.

Entrada do programa para a funcionalidade [11]:

```
11  arquivoEntrada.bin  nomeEstacaoOrigem  valorOrigem  nomeEstacaoDestino
valorDestino
```

onde:

- `estacao.bin` é um arquivo binário gerado conforme as especificações descritas no primeiro trabalho prático.
- `nomeEstacaoOrigem` é o nome do campo.
- `valorOrigem` é o valor passado como entrada para o `nomeEstacaoOrigem`. Deve ser colocado aspas duplas por se tratar de uma *string*.
- `nomeEstacaoDestino` é o nome do campo.
- `valorDestino` é o valor passado como entrada para o `nomeEstacaoDestino`. Deve ser colocado aspas duplas por se tratar de uma *string*.

Saída caso o programa seja executado com sucesso:

A saída deve ser exibida na saída padrão da seguinte forma. Primeiramente, escrever em uma linha o número de estações que serão percorridas. Esse número de estações não deve considerar a estação de origem, mas deve considerar a estação de destino. Depois, escrever em outra linha a distância que será percorrida. Na sequência, escrever em outra linha os valores para os seguintes campos: o nome da estação de origem, os nomes das estações intermediárias e o nome da estação de destino. Devem ser escritos uma vírgula e um espaço em branco entre cada valor.

Exemplo com metadados:

Numero de estacoes que serao percorridas: x

Distancia que sera percorrida: y

nomeEstOrigem, nomeEstacao1, nomeEstacao2, ..., nomeEstDestino

OBS: o valor de x não inclui a estação de origem, mas inclui a estação de destino

Mensagem de saída caso não exista um caminho entre as estações solicitadas:

Não existe caminho entre as estações solicitadas.

Mensagem de saída caso algum erro seja encontrado:

Falha na execução da funcionalidade.

Exemplo de execução:

```
./programaTrab
```

```
11 estacao.bin nomeEstacaoOrigem "Luz" nomeEstacaoDestino "Oscar  
Freire"
```

Numero de estacoes que serao percorridas: 4

Distancia que sera percorrida: 4790

Luz, Republica, Higienopolis-Mackenzie, Paulista, Oscar Freire

[12] Você foi contratado como um gestor de uma cidade inteligente e, como primeira atividade, ficou responsável por melhorar o projeto da malha de metrô da cidade. Nessa melhoria, foi solicitado que você gastasse a menor quantia possível para projetar novamente a malha, mas garantindo que as pessoas possam sair de qualquer estação e chegar em qualquer outra estação. Você também foi informado que as linhas podem ser de ida e volta, indicando que se existe uma linha entre uma estação 1 e uma estação 2, as pessoas podem ir da estação 1 para a estação 2 e vice-versa. Para desenvolver o seu projeto, foi passado a você a malha de metrô existente, com suas estações, linhas e distâncias existentes. Como resultado, você deve produzir a sua resposta na forma de listas de adjacências, seguindo o mesmo formato definido na funcionalidade [10]. Considere que, durante a execução do algoritmo, qualquer decisão a respeito da ordem de inserção ou análise dos vértices/arestas deve ser feita da seguinte forma: (i) em situação de empate na escolha de vértices, deve ser escolhido o vértice cujo valor do nome da estação seja o menor; e (ii) em situação de empate entre o peso de duas arestas (u_1, v_1) e (u_2, v_2) , deve ser escolhida a aresta cujo valor de u seja o menor. Caso haja situação de empate entre u_1 e u_2 , deve ser escolhida a aresta cujo valor de v seja o menor. Todos os índices dos vetores e listas devem ser ordenados de forma crescente de acordo com o nome da estação.

Sintaxe do comando para a funcionalidade [12]:

```
12 arquivoEntrada.bin nomeEstacaoOrigem "valorOrigem"
```

onde:

- arquivoEntrada.bin é um arquivo binário gerado conforme as especificações descritas no primeiro trabalho prático.
- nomeEstacaoOrigem é o nome do campo.
- valorOrigem é o valor passado como entrada para o nomeEstacaoOrigem. Deve ser colocado aspas duplas por se tratar de uma *string*.

Saída caso o programa seja executado com sucesso:

A saída deve ser exibida na saída padrão da seguinte forma. Percorra a árvore geradora mínima utilizando o algoritmo de busca em profundidade a partir do vértice passado como entrada, o qual é representado pelo valorOrigem do atributo nomeEstacaoOrigem. Em cada chamada recursiva, escreva o nome da estação relativa ao vértice atual, chame recursivamente o algoritmo para o filho deste vértice e continue o processo até que todos vértices tenham sido analisados. O formato deve seguir a seguinte ordem: nome da estação relativa ao vértice atual, nome da estação relativa ao vértice filho, distância. Na próxima linha: nome da estação relativa ao vértice atual, nome da estação relativa ao vértice filho, distância, e assim sucessivamente. Caso um vértice tenha mais do que um filho, os filhos devem ser ordenados em forma crescente de acordo com o nome da estação.

Exemplo com metadados:

```
nomeEstacao11, nomeEstacao12, distancia1  
nomeEstacao21, nomeEstacao22, distancia2  
...  
nomeEstacaon1, nomeEstacaon2, distancian
```

Mensagem de saída caso algum erro seja encontrado:

Falha na execução da funcionalidade.

Exemplo de execução (são mostrados apenas alguns elementos):

```
./programaTrab  
12 estacao.bin nomeEstacaoOrigem "Engenheiro Manoel Feio"  
Engenheiro Manoel Feio, Itaquaquecebuta, 2270  
Engenheiro Manoel Feio, Jardim Romando, 1990  
Jardim Romano, Itaim Paulista, 2060  
...
```


[13] Suponha que você está em uma determinada estação origem e decide fazer diversos passeios, sempre saindo dessa estação origem e voltando a ela. Para tanto, você quer saber quantos ciclos simples existem a partir dessa estação origem. Um ciclo simples é um ciclo em que nenhum vértice se repete, com exceção do primeiro e do último (ou seja, o primeiro vértice, chamado de vértice de origem, é o mesmo que o último vértice, chamado de vértice de destino).

Entrada do programa para a funcionalidade [13]:

```
13 arquivoEntrada.bin nomeEstacaoOrigem "valorOrigem"
```

onde:

- arquivoEntrada.bin é um arquivo binário gerado conforme as especificações descritas no primeiro trabalho prático.
- nomeEstacaoOrigem é o nome do campo.
- valorOrigem é o valor passado como entrada para o nomeEstacaoOrigem. Deve ser colocado aspas duplas por se tratar de uma *string*.

Saída caso o programa seja executado com sucesso:

Escreva primeiro a *string* "Quantidade de ciclos:" deixe um espaço em branco e em seguida escreva a quantidade de ciclos simples existentes no grafo a partir da estação de origem definida como entrada. Caso não seja possível voltar à estação de origem, escreva -1.

Mensagem de saída caso algum erro seja encontrado:

Falha na execução da funcionalidade.

Exemplos de execução:

```
./programaTrab  
13 estacao.bin nomeEstacaoOrigem "Engenheiro Manoel Feio"  
Quantidade de ciclos: 3  
  
./programaTrab  
13 estacao.bin nomeEstacaoOrigem "Jardim Romando"  
Quantidade de ciclos: -1
```

Restrições

As seguintes restrições têm que ser garantidas no desenvolvimento do trabalho.

[1] O arquivo de dados deve ser gravado em disco no **modo binário**. O modo texto não pode ser usado.

[2] Os dados do registro descrevem os nomes dos campos, os quais não podem ser alterados. Ademais, todos os campos devem estar presentes na implementação, e nenhum campo adicional pode ser incluído. O tamanho e a ordem de cada campo deve obrigatoriamente seguir a especificação.

[3] Deve haver a manipulação de valores nulos, conforme as instruções definidas.

[4] Não é necessário realizar o tratamento de truncamento de dados.

[5] Devem ser exibidos avisos ou mensagens de erro de acordo com a especificação de cada funcionalidade.

[6] Os dados devem ser obrigatoriamente escritos campo a campo. Ou seja, não é possível escrever os dados registro a registro. Essa restrição refere-se à entrada/saída, ou seja, à forma como os dados são escritos no arquivo.

[7] O(s) aluno(s) que desenvolveu(desenvolveram) o trabalho prático deve(m) constar como comentário no início do código (i.e. NUSP e nome do aluno). Para trabalhos desenvolvidos por mais do que um aluno, não será atribuída nota ao aluno cujos dados não constarem no código fonte.

[8] Todo código fonte deve ser documentado. A **documentação interna** inclui, dentre outros, a documentação de procedimentos, de funções, de variáveis, de partes do código fonte que realizam tarefas específicas. Ou seja, o código fonte deve ser

documentado tanto em nível de rotinas quanto em nível de variáveis e blocos funcionais.

[9] A implementação deve ser realizada usando as linguagens de programação C e C++. As funções das bibliotecas `<stdio.h>` devem ser utilizadas para operações relacionadas à escrita e leitura dos arquivos. A implementação não pode ser feita em qualquer outra linguagem de programação. O programa executará no [run.codes].

Material para Entregar

Arquivo compactado (a ser entregue no *run.codes*)

Deve ser preparado um arquivo .zip contendo:

- Código fonte do programa devidamente documentado.
- Makefile para a compilação do programa.

Vídeo (a ser entregue no *e-disciplinas*)

- Um vídeo gravado pelos integrantes do grupo, o qual deve ter, no máximo, 7 minutos de gravação. O vídeo deve explicar o trabalho desenvolvido. Ou seja, o grupo deve apresentar: cada funcionalidade e uma breve descrição de como a funcionalidade foi implementada. Todos os integrantes do grupo devem participar do vídeo, sendo que o tempo de apresentação dos integrantes deve ser balanceado. Ou seja, o tempo de participação de cada integrante deve ser aproximadamente o mesmo. O uso da webcam é obrigatório.

Instruções para fazer o arquivo makefile. No [run.codes] tem uma orientação para que, no makefile, a diretiva “all” contenha apenas o comando para compilar seu programa e, na diretiva “run”, apenas o comando para executá-lo. Adicionalmente, para utilizar a função `binarioNaTela`, é necessário usar a flag `-lmd`. Assim, a forma mais simples de se fazer o arquivo makefile é:

all:

```
gcc -o programaTrab *.c -lmd  
run:  
./programaTrab
```

Lembrando que *.c já engloba todos os arquivos .c presentes no arquivo zip. Adicionalmente, no arquivo Makefile é importante se ter um *tab* nos locais colocados acima, senão ele pode não funcionar.

Instruções de entrega.

O programa deve ser submetido via [run.codes]:

- página: <https://runcodes.icmc.usp.br/>
- Código de matrícula: **L6SW**

O vídeo gravado deve ser submetido por meio da página da disciplina no e-disciplinas, no qual o grupo vai informar o nome de cada integrante, o número do grupo e um link que contém o vídeo gravado. Ao submeter o link, verifique se o mesmo pode ser acessado. Vídeos cujos links não puderem ser acessados receberão nota zero. Vídeos corrompidos ou que não puderem ser corretamente acessados receberão nota zero.

Critério de Correção

Critério de avaliação do trabalho. Na correção do trabalho, serão ponderados os seguintes aspectos.

- Corretude da execução do programa.
- Atendimento às especificações do registro de cabeçalho e dos registros de dados.
- Atendimento às especificações da sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade.
- Qualidade da documentação entregue. A documentação interna terá um peso considerável no trabalho.

- Estruturação da solução. O projeto deve apresentar uma organização adequada de arquivos e diretórios.
- Modularização. Trechos de código que aparecem repetidamente devem ser modularizados por meio de funções ou procedimentos, evitando duplicação de código.
- Uso de ferramentas de inteligência artificial. Não deve ser realizado para a implementação das funcionalidades especificadas neste trabalho e outras funções julgadas como necessárias para o embasamento do conteúdo ministrado. O uso dessas ferramentas acarretará desconto rigoroso na nota.
- Vídeo. Integrantes que não participarem da apresentação receberão nota 0 no trabalho correspondente.

Casos de teste no [run.codes]. Juntamente com a especificação do trabalho, serão disponibilizados 70% dos casos de teste no [run.codes], para que os alunos possam avaliar o programa sendo desenvolvido. Os 30% restantes dos casos de teste serão utilizados nas correções.

Restrições adicionais sobre o critério de correção.

- O uso excessivo de ferramentas de inteligência artificial acarretará a atribuição de nota zero (0) ao trabalho.
- A não execução de um programa devido a erros de compilação implica que a nota final da parte do trabalho será igual a zero (0).
- O não atendimento às especificações do registro de cabeçalho e dos registros de dados implica que haverá uma diminuição expressiva na nota do trabalho.
- O não atendimento às especificações de sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade implica que haverá uma diminuição expressiva na nota do trabalho.
- A ausência da documentação implica que haverá uma diminuição expressiva na nota do trabalho.
- A realização do trabalho prático com alunos de turmas diferentes implica que haverá uma diminuição expressiva na nota do trabalho.

- A inserção de palavras ofensivas nos arquivos e em qualquer outro material entregue implica que a nota final da parte do trabalho será igual a zero (0).
- Em caso de plágio, as notas dos trabalhos envolvidos serão zero (0).

Bom Trabalho!